Stacked DRG 상세 분석

개요

Stacked Depth Robust Graph(적층 깊이 견고 그래프)는 파일코인 PoRep의 핵심 구조입니다. 이 문서에서는 데이터가 노드로 표현되는 그래프 구조와 그 위에서 수행되는 복제 과정을 상세히 분석합니다.

목차

- 핵심 개념: 데이터 = 노드
- DRG 기본 개념
- Stacked DRG 구조
- 부모 노드 생성 메커니즘
- 레이어별 인코딩 과정
- 그래프 샘플링과 BucketGraph
- 보안 분석
- 성능 최적화
- 코드 구현 분석

핵심 개념: 데이터 = 노드

데이터의 노드 표현

파일코인에서 저장할 데이터는 **그래프의 노드(Node)**로 표현됩니다. 이는 단순한 추상화가 아니라 실제 구현의 핵심입니다.

```
원본 데이터 (32GiB 섹터)
↓
32바이트 청크로 분할
↓
각 청크 = 하나의 노드
↓
총 노드 수 = 2^30개 (약 10억 개)
```

노드의 구체적 의미

1. 물리적 표현

- **노드 ID**: 0부터 N-1까지의 정수 (N = 총 노드 수)
- **노드 데이터**: 32바이트 크기의 데이터 청크
- **노드 위치**: 파일 내에서의 오프셋 (node_id × 32 바이트)

2. 논리적 표현

```rust
struct Node {
id: u64, // 노드 식별자
data: [u8; 32], // 실제 데이터 (32바이트)
parents: Vec<u64>, // 부모 노드들의 ID
label: [u8; 32], // 계산된 레이블
```

```
}
```

3. 그래프에서의 역할

- **정점(Vertex)**: 그래프 이론에서 노드는 정점
- **간선(Edge)**: 부모-자식 관계가 간선
- **연결성**: 각 노드는 여러 부모 노드와 연결

데이터 처리 흐름

```
1. 원본 데이터 입력
├── 32GiB 파일
└── 1,073,741,824개 노드로 분할

2. 각 노드별 처리
├── 부모 노드 선택 (DRG + 확장)
├── 레이블 계산 (부모들의 해시)
└── 인코딩 (레이블 ⊕ 원본 데이터)

3. 레이어별 반복
├── 레이어 0: 원본 데이터
├── 레이어 1-10: 인코딩된 데이터
└── 각 레이어마다 모든 노드 재계산
```

DRG 기본 개념

Depth Robust Graph의 정의

DRG는 **깊이 견고성(Depth Robustness)**을 가진 그래프입니다.

수학적 정의

- **그래프 G = (V, E)**: V는 노드 집합, E는 간선 집합
- **깊이 견고성**: 임의의 부분 그래프를 제거해도 남은 그래프의 깊이가 보장됨
- **확장성(Expansion)**: 모든 노드 부분집합에 대해 충분한 이웃 노드 존재

파일코인에서의 구체적 매개변수

```
const BASE_DEGREE: usize = 6; // 기본 부모 수
const EXP_DEGREE: usize = 8; // 확장 부모 수
const TOTAL_PARENTS: usize = 14; // 총 부모 수
```

DRG의 보안 특성

1. 시간-공간 트레이드오프 방지

```
공격자가 저장 공간을 줄이려면
                    ↓
일부 노드의 데이터를 삭제
                    ↓
해당 노드 재계산 시 부모 노드들 필요
                    ↓
부모 노드들도 재계산 필요 (연쇄 반응)
                    ↓
```

```
결국 전체 계산 시간 증가
```

2. 병렬 처리 제한

- DRG의 깊이가 깊어 순차적 계산 강제
- 각 노드는 부모 노드들의 계산 완료 후에만 계산 가능
- 메모리 하드 함수(Memory-Hard Function) 특성

Stacked DRG 구조

기본 DRG vs Stacked DRG

기본 DRG의 한계

```
기본 DRG만으로는
├── 보안성 부족 (공격 가능성 존재)
├── 확장성 제한 (큰 데이터 처리 어려움)
└── 효율성 문제 (과도한 메모리 사용)
```

Stacked DRG의 해결책

```
Stacked DRG = 기본 DRG + 확장 부모 + 레이어 구조
├── 11개 레이어로 구성
├── 각 레이어마다 전체 노드 재인코딩
└── 레이어 간 의존성으로 보안성 강화
```

레이어 구조 상세 분석

레이어 0 (원본 데이터)

```
// 레이어 0은 원본 데이터 그대로
layer_0[node_id] = original_data[node_id]
```

레이어 1-10 (인코딩된 레이어)

```
// 각 레이어는 이전 레이어를 기반으로 인코딩
for layer in 1..=10 {
for node_id in 0..total_nodes {
        // 1. 부모 노드들 선택
        let parents = select_parents(node_id, layer);

        // 2. 레이블 계산
        let label = create_label(parents, layer, node_id);

        // 3. 인코딩 수행
        layer[layer_id][node_id] = label ⊕ previous_layer[node_id];
        }
}
```

그래프 구조의 시각화

```
레이어 구조:

Layer 0: [Node0] [Node1] [Node2] ... [NodeN] (원본)

                        ↓               ↓             ↓

Layer 1: [Node0] [Node1] [Node2] ... [NodeN] (1차 인코딩)

                        ↓               ↓              ↓

Layer 2: [Node0] [Node1] [Node2] ... [NodeN] (2차 인코딩)

                                                  ...

Layer 10: [Node0] [Node1] [Node2] ... [NodeN] (최종 인코딩)



각 노드의 부모 관계:

Node[i] ← Base Parents (6개) + Expansion Parents (8개)

├── DRG Parents: 규칙적 선택

└── Exp Parents: 의사무작위 선택
```

부모 노드 생성 메커니즘

기본 부모 (Base Parents) 생성

DRG 기반 부모 선택

```rust
fn select_base_parents(node_id: u64, total_nodes: u64) -> Vec<u64> {
    let mut parents = Vec::new();
```

```
        // 1. 직전 노드 (Sequential Parent)
        if node_id > 0 {
            parents.push(node_id - 1);
        }

        // 2. DRG 부모들 (5개)
        for i in 1..=5 {
            let parent = calculate_drg_parent(node_id, i, total_nodes);
            parents.push(parent);
        }

    parents
    }

    fn calculate_drg_parent(node_id: u64, parent_index: u32, total_nodes: u64) -> u
        // BucketGraph 알고리즘 사용
        let bucket_size = total_nodes / BASE_DEGREE;
        let bucket_id = parent_index as u64;
        let offset = hash_function(node_id, parent_index) % bucket_size;

        (bucket_id * bucket_size + offset) % node_id.max(1)
    }
```

확장 부모 (Expansion Parents) 생성

Feistel 암호 기반 생성

```rust
fn select_expansion_parents(
node_id: u64,
layer: u32,
replica_id: &[u8; 32]
) -> Vec<u64> {
        let mut parents = Vec::new();

        for i in 0..EXP_DEGREE {
                // Feistel 네트워크를 사용한 의사무작위 생성
                let parent = feistel_permutation(
                        node_id,
                        layer,
                        i,
                        replica_id
                );
                parents.push(parent);
        }

        parents
}
```

```
fn feistel_permutation(
node_id: u64,
layer: u32,
parent_index: u32,
replica_id: &[u8; 32]
) -> u64 {
        // Feistel 네트워크 구현
        let mut left = (node_id >> 32) as u32;
        let mut right = node_id as u32;

        // 여러 라운드의 Feistel 변환
        for round in 0..4 {
                let round_key = derive_round_key(replica_id, layer, parent_inde
                let temp = left;
                left = right;
                right = temp ^ feistel_function(right, round_key);
        }

        ((left as u64) << 32) | (right as u64)
}
```

부모 선택의 보안적 의미

1. 기본 부모의 역할

- **구조적 연결성**: 그래프의 기본 연결 구조 제공
- **순차적 의존성**: 이전 노드들과의 연결로 순차 계산 강제
- **예측 가능성**: 결정론적 선택으로 검증 가능

2. 확장 부모의 역할

- **무작위성 추가**: 예측 불가능한 연결 패턴
- **공격 방어**: 특정 패턴 공격 방지
- **보안 강화**: 추가적인 계산 복잡도 제공

레이어별 인코딩 과정

레이블 생성 (Label Creation)

create_label 함수의 핵심 로직

```
fn create_label<H: Hasher>(
parents: &[u64],
layer: u32,
node_id: u64,
replica_id: &H::Domain,
data: &[u8]
) -> Result<H::Domain> {
        // 1. 해시 입력 준비
        let mut hasher = H::new();
```

```
        // 2. 복제본 ID 추가
        hasher.update(AsRef::<[u8]>::as_ref(replica_id));

        // 3. 레이어 정보 추가
        hasher.update(&(layer  as u64).to_be_bytes());

        // 4. 노드 ID 추가
        hasher.update(&node_id.to_be_bytes());

        // 5. 모든 부모 노드의 데이터 추가
        for &parent_id  in  parents {
                let  parent_data = get_node_data(parent_id, layer - 1, data);
                hasher.update(parent_data);
}

// 6. 최종 해시 계산
Ok(hasher.finalize())
}
```

인코딩 과정 (Encoding Process)

각 노드의 인코딩

```
fn encode_node(
node_id: u64,
layer: u32,
original_data: &[u8; 32],
label: &[u8; 32]
) -> [u8; 32] {
        // XOR 연산으로 인코딩
        let mut encoded = [0u8; 32];
        for i in 0..32 {
                encoded[i] = original_data[i] ^ label[i];
        }
        encoded
}
```

전체 레이어 인코딩

```
fn encode_layer(
layer: u32,
previous_layer_data: &[[u8; 32]],
replica_id: &[u8; 32],
total_nodes: u64
) -> Vec<[u8; 32]> {
        let mut encoded_layer = Vec::with_capacity(total_nodes as usize);
```

```
for node_id in 0..total_nodes {
        // 1. 부모 노드들 선택
        let base_parents = select_base_parents(node_id, total_nodes);
        let exp_parents = select_expansion_parents(node_id, layer, repl
        let all_parents = [base_parents, exp_parents].concat();

        // 2. 레이블 생성
        let label = create_label(
                &all_parents,
                layer,
                node_id,
                replica_id,
                previous_layer_data
        );

        // 3. 인코딩 수행
        let encoded_data = encode_node(
                node_id,
                layer,
                &previous_layer_data[node_id as usize],
                &label
        );
```

```
                encoded_layer.push(encoded_data);
        }

        encoded_layer
}
```

그래프 샘플링과 BucketGraph

BucketGraph 알고리즘

개념

BucketGraph는 균등한 차수 분포를 보장하는 그래프 샘플링 알고리즘입니다.

```
struct BucketGraph {
        total_nodes: u64,
        degree: usize,
        bucket_size: u64,
}

impl BucketGraph {
fn  new(total_nodes: u64, degree: usize) -> Self {
        Self {
                total_nodes,
```

```
                degree,
                bucket_size: total_nodes / degree  as u64,
        }
    }

    fn  sample_parents(&self, node_id: u64) -> Vec<u64> {
        let  mut  parents = Vec::new();

        for  bucket_id  in  0..self.degree {
                let  bucket_start = bucket_id  as u64 * self.bucket_size;
                let  bucket_end = bucket_start + self.bucket_size;

                // 각 버킷에서 하나씩 샘플링
                let  offset = self.hash_to_bucket_offset(node_id, bucket_id);
                let  parent = (bucket_start + offset) % node_id.max(1);

                parents.push(parent);
        }

        parents
    }

    fn  hash_to_bucket_offset(&self, node_id: u64, bucket_id: usize) -> u64 {
```

```
        // 해시 함수를 사용해 버킷 내 오프셋 계산
        let mut hasher = Blake2s::new();
        hasher.update(&node_id.to_be_bytes());
        hasher.update(&(bucket_id as u64).to_be_bytes());
        let hash = hasher.finalize();

        u64::from_be_bytes([
                hash[0], hash[1], hash[2], hash[3],
                hash[4], hash[5], hash[6], hash[7]
        ]) % self.bucket_size
        }
}
```

샘플링의 균등성 보장

문제점

```
무작위 샘플링의 문제:
├── 일부 노드가 너무 많은 자식을 가질 수 있음
├── 일부 노드가 전혀 참조되지 않을 수 있음
└── 불균등한 계산 부하 분산
```

BucketGraph의 해결책

```
버킷 기반 샘플링:
├── 전체 노드를 degree개의 버킷으로 분할
├── 각 버킷에서 정확히 하나씩 선택
└── 모든 노드가 균등하게 참조됨
```

보안 분석

시간-공간 트레이드오프 공격 방어

공격 시나리오

```
공격자의 목표:
├── 저장 공간 절약 (일부 데이터 삭제)
├── 필요시 빠른 재계산
└── 정당한 증명 생성
```

Stacked DRG의 방어 메커니즘

```
방어 전략:
├── 깊은 의존성 체인
│   ├── 각 노드는 14개 부모에 의존
│   ├── 부모들도 각각 14개 부모에 의존
│   └── 연쇄적 재계산 필요
├── 레이어별 의존성
```

```
│ ├── 11개 레이어 순차 계산 필요
│ ├── 각 레이어는 이전 레이어에 의존
│ └── 병렬 처리 제한
└── 메모리 하드 특성
├── 중간 결과 저장 필요
├── 메모리 사용량 최소화 불가
└── 계산 시간 vs 저장 공간 트레이드오프
```

그라인딩 공격 방어

공격 방법

```
그라인딩 공격:
├── 유리한 challenge 찾기 위한 반복 시도
├── replica_id 조작을 통한 그래프 구조 변경
└── 계산하기 쉬운 구조 탐색
```

방어 메커니즘

```
방어 요소:
├── replica_id의 무작위성
│ ├── 외부에서 제공되는 무작위 값
│ ├── 조작 불가능한 구조
│ └── 모든 그래프 생성에 영향
├── Feistel 네트워크
```

```
│ ├── 암호학적으로 안전한 순열
│ ├── 역함수 계산 어려움
│ └── 구조 예측 불가능
└── 다층 보안
├── 기본 부모 + 확장 부모
├── 여러 해시 함수 조합
└── 레이어별 독립적 보안
```

성능 최적화

부모 캐싱 시스템

캐시 구조

```rust
struct ParentCache {
// 기본 부모들 (예측 가능하므로 사전 계산)
base_parents: Vec<Vec<u64>>,

// 확장 부모들 (레이어별로 다름)
expansion_parents: HashMap<(u32, u64), Vec<u64>>,

// LRU 캐시로 메모리 관리
lru_cache: LruCache<u64, Vec<u64>>,
}
```

```
impl ParentCache {
fn get_parents(&mut self, node_id: u64, layer: u32) -> Vec<u64> {
// 1. 캐시에서 확인
        if let Some(parents) = self.lru_cache.get(&node_id) {
                return parents.clone();
                }

                // 2. 기본 부모 가져오기
                let mut parents = self.base_parents[node_id as usize].clone()

                // 3. 확장 부모 계산 또는 캐시에서 가져오기
                let exp_key = (layer, node_id);
                let exp_parents = self.expansion_parents
                .entry(exp_key)
                .or_insert_with(|| self.calculate_expansion_parents(node_id, lay

                parents.extend_from_slice(exp_parents);

                // 4. 캐시에 저장
                self.lru_cache.put(node_id, parents.clone());

                parents
```

```
        }
}
```

메모리 최적화

스트리밍 처리

```
fn encode_layer_streaming(
layer: u32,
input_path: &Path,
output_path: &Path,
replica_id: &[u8; 32],
total_nodes: u64
) -> Result<()> {
        const CHUNK_SIZE: usize = 1024 * 1024; // 1MB 청크

        let mut input_file = File::open(input_path)?;
        let mut output_file = File::create(output_path)?;

        for chunk_start in (0..total_nodes).step_by(CHUNK_SIZE) {
                let chunk_end = (chunk_start + CHUNK_SIZE as u64).min(total_nod

                // 청크 단위로 데이터 읽기
                let mut input_chunk = vec![[0u8; 32]; (chunk_end - chunk_start
```

```
            input_file.read_exact(bytemuck::cast_slice_mut(&mut  input_chunk


            // 청크 내 노드들 병렬 처리
            let  output_chunk: Vec<[u8; 32]> = input_chunk
            .par_iter()
            .enumerate()
            .map(|(i, &node_data)| {
            let  node_id = chunk_start + i  as u64;
            let  parents = get_parents(node_id, layer, replica_id);
            let  label = create_label(&parents, layer, node_id, replica_id);
            encode_node(node_id, layer, &node_data, &label)
            })
            .collect();


            // 결과 쓰기
            output_file.write_all(bytemuck::cast_slice(&output_chunk))?;
        }


        Ok(())
    }
```

병렬 처리 최적화

노드별 병렬 처리

```
use rayon::prelude::*;


fn encode_layer_parallel(
layer: u32,
previous_layer: &[[u8; 32]],
replica_id: &[u8; 32]
) -> Vec<[u8; 32]> {
        (0..previous_layer.len())
        .into_par_iter()
        .map(|node_id| {
                let parents = get_parents(node_id as u64, layer, replica_id);
                let label = create_label(&parents, layer, node_id as u64, repl
                encode_node(node_id as u64, layer, &previous_layer[node_id], &l
        })
        .collect()
}
```

코드 구현 분석

핵심 구조체

StackedBucketGraph

```rust
pub struct StackedBucketGraph<H: Hasher> {
base_graph: BucketGraph<H>,
expansion_degree: usize,
nodes: usize,
_h: PhantomData<H>,
}


impl<H: Hasher> StackedBucketGraph<H> {
        pub fn new_stacked(
        nodes: usize,
        base_degree: usize,
        expansion_degree: usize,
        porep_id: &H::Domain,
        ) -> Result<Self> {
                Ok(Self {
                base_graph: BucketGraph::new(nodes, base_degree, porep_id)?,
                expansion_degree,
                nodes,
                _h: PhantomData,
                })
        }
```

```
        pub fn parents(&self, node: usize, layer: Option<u32>) -> Result<Vec<u
            let mut parents = self.base_graph.parents(node)?;

            if let Some(layer_id) = layer {
                let exp_parents = self.expansion_parents(node, layer_id
                parents.extend(exp_parents);
            }

            Ok(parents)
        }

        fn expansion_parents(&self, node: usize, layer: u32) -> Result<Vec<usiz
            let mut parents = Vec::with_capacity(self.expansion_degree);

            for i in 0..self.expansion_degree {
                let parent = self.expanded_parent(node, i, layer)?;
            parents.push(parent);
            }

            Ok(parents)
        }
    }
```

레이블 생성 구현

create_label_exp 함수

```
pub fn create_label_exp<H: Hasher>(
graph: &StackedBucketGraph<H>,
parents: &[u32],
layer: u32,
node: u64,
porep_id: &H::Domain,
) -> Result<H::Domain> {
        let mut hasher = H::new();

        // porep_id 추가
        hasher.update(AsRef::<[u8]>::as_ref(porep_id));

        // layer 정보 추가
        hasher.update(&(layer as u64).to_be_bytes());

        // node 정보 추가
        hasher.update(&node.to_be_bytes());

        // 모든 부모의 데이터 추가
        for &parent in parents {
```

```
            let parent_data = get_node_data(parent as u64, layer)?;
            hasher.update(&parent_data);
        }

        Ok(hasher.finalize())
}
```

실제 사용 예제

전체 인코딩 과정

```
fn full_encoding_example() -> Result<()> {
        const NODES: usize = 1024; // 예제용 작은 크기
        const LAYERS: u32 = 11;

        // 1. 그래프 생성
        let porep_id = [0u8; 32];
        let graph = StackedBucketGraph::<Blake2sHasher>::new_stacked(
        NODES,
        BASE_DEGREE,
        EXP_DEGREE,
        &porep_id,
        )?;
```

```
// 2. 원본 데이터 준비
let mut current_layer: Vec<[u8; 32]> = (0..NODES)
.map(|i| {
let mut data = [0u8; 32];
data[..8].copy_from_slice(&(i as u64).to_be_bytes());
data
})
.collect();

// 3. 레이어별 인코딩
for layer in 1..=LAYERS {
println!("Processing layer {}", layer);

let next_layer: Vec<[u8; 32]> = (0..NODES)
.map(|node_id| {
// 부모 노드들 가져오기
let parents = graph.parents(node_id, Some(layer))?;

// 레이블 생성
let label = create_label_exp(
&graph,
&parents.iter().map(|&p| p as u32).collect::<Vec<_>>(),
layer,
```

```
        node_id  as u64,
        &porep_id,
        )?;

        // 인코딩
        let  mut  encoded = current_layer[node_id];
        for (i, &label_byte) in  label.as_ref().iter().enumerate() {
                encoded[i] ^= label_byte;
        }

        Ok(encoded)
        })
        .collect::<Result<Vec<_>>>()?;

        current_layer = next_layer;
        }

        println!("Encoding completed!");
        Ok(())
}
```

결론

Stacked DRG는 파일코인 PoRep의 핵심 구조로, 다음과 같은 특징을 가집니다:

핵심 특징

1. **데이터 = 노드**: 모든 데이터는 그래프의 노드로 표현
2. **계층적 구조**: 11개 레이어의 순차적 인코딩
3. **복합 부모 시스템**: 기본 부모 + 확장 부모
4. **보안성**: 시간-공간 트레이드오프 공격 방어
5. **효율성**: 캐싱과 병렬 처리를 통한 최적화

보안 보장

- **메모리 하드**: 저장 공간 절약 시 계산 시간 증가
- **순차 처리**: 병렬 처리 제한으로 공정한 경쟁
- **무작위성**: 예측 불가능한 그래프 구조

구현 복잡성

- **그래프 생성**: BucketGraph와 Feistel 네트워크
- **레이블 계산**: 암호학적 해시 함수 활용
- **메모리 관리**: 스트리밍과 캐싱 기법

이러한 설계를 통해 Stacked DRG는 파일코인의 저장 증명 시스템에서 핵심적인 역할을 수행하며, 공격자로부터 네트워크를 보호하는 동시에 효율적인 증명 생성을 가능하게 합니다.